onedoor

User Manual — the policy engine and the Policy Studio

"The model proposes; the policy layer disposes."

Covers onedoor 0.6.2 (published) and the Policy Studio v2 (ships as 0.7.0). August 2026 · internal preview edition — the v2 Studio screens described here are built and tested but not yet publicly released.

1 · What onedoor is

onedoor is a tiered guardrail engine for agentic systems. Every action your software wants to take — scheduled, rule-fired, LLM-proposed, or human-clicked — is submitted as a structured **ActionRequest** and evaluated by one decision pipeline before anything touches the world. Nothing executes around it; that is the name.

The three-minute mental model

One door	Callers never execute directly; they submit an ActionRequest (action type + parameters + rationale) to the engine.
Ordered checks	Kill switch → policy lookup (default-deny for unknown action types) → tier-1 integrity (no undo → no autonomy) → bounds → dry-run → caps (reserved race-free) → intent recorded.
Four outcomes	*Permitted* (an obligation: you enforce, then report) · *denied* (with a typed reason) · *proposed* (waiting for a human approval) · *dry-run* (logged, nothing happens).
Everything audited	An append-only log: decisions, results, denials, approvals, kill-switch blocks — every row carries a reason code.

> **The most load-bearing fact:** an action type you have not written a policy for cannot auto-execute. Absence is denial — unknown actions land at Tier 3, propose-and-confirm. An empty policy table means *nothing is permitted*, and the Studio says so on its face.

2 · Core concepts

Autonomy tiers

Tier	Name	Meaning
0	Observe	Audited reads. Logged, allowed, never reach connectors; exempt from the kill switch.
1	Auto	Auto-executes — only if reversible (a compensating command exists), in bounds, with an undo window.
2	Auto-capped	Auto-executes under caps: a rate per day plus cumulative money budgets (per day / per month).
3	Confirm	Propose-and-confirm: a human approves, with a TTL. Where every unknown action type lands.

Tiers attach to **action types**, not to agents. There is deliberately no deny tier: denial is the default, and a rule's job is to grant, never to forbid.

Drafts vs. policy versions

A **draft** is a policy set that has not been ratified: the only editable, non-binding object in the system — a bill, not a law. The engine never reads drafts. Ratifying a draft (through the Studio's ceremony) turns it into a **numbered, immutable policy version** — the thing the engine enforces and receipts attest. Versions are never edited, only succeeded: to change policy, draft again and ratify version n+1; version n stands in the record forever. Everything mutable has no authority; everything with authority is immutable.

Receipts

Every ratification produces a receipt: a small record binding the ratified version to a content digest. A receipt can be verified **offline**, with a command that reads two files and opens no database (§8). The Studio shows you a verification and tells you how to repeat it without trusting the Studio.

3 · Getting started

Install (Windows paths shown; on macOS/Linux use .venv/bin/…)

```
python -m venv .venv
.venv\Scripts\pip install -e ".[studio]"     # from a repo checkout
# or, from PyPI:
.venv\Scripts\pip install "onedoor[studio]"
```

Check, then start the Studio

```
python -m onedoor.studio --help
python -m onedoor.studio --db onedoor.db --studio-db studio.db --port 8787
```

Then open **http://127.0.0.1:8787**.

> **Two things to know before anything else.** (1) **--host accepts loopback and nothing else** — there is no flag to expose the Studio to a network, by design. (2) **Name --db explicitly.** The decision service defaults to *onedoor-service.db* and the Studio to *onedoor.db*; accept both defaults and they point at different stores — the Studio comes up, works, and shows you an empty world. A fresh store says so on its face, naming both defaults.

4 · The Policy Studio — the eight screens

Screen	What it is for
Drafts	Where policy sets are born. The landing page; even empty it offers a form and a curl one-liner.
Policies	What is actually in force. Empty reads “nothing is permitted” — absence is denial.
History	The execution ledger: every decision, chain-numbered so a citation survives the view, with the policy version that made it and its reason code.
Live state	Budgets (consumed = counter – reserved) and the kill switch — shown here, thrown only via the admin API (capability is not authority), with what the switch does *not* stop stated on the page.
Editor	Guided pane and raw pane rendered from one server-side parser — they cannot disagree. Saving nonsense refuses, says the draft is unchanged, and writes nothing.
Ceremony	Ratification. A page before it is an action: what will be in force, what changes, what ratifying does not undo — in the ceremony's own words.
Replay	Open any past decision and re-evaluate it under a different policy version. Calls the real engine and answers in the conditional: what *would have* happened.
Verify	The deposition page — method first, answer last: the receipt, the two files, and the offline command that checks them without trusting the Studio.

5 · Writing policy

Policies are data, not code. Under the hood a policy set is YAML; the Studio's editor reads and writes the same object its raw pane shows. A minimal rule binds an action type to a tier; a production rule usually carries reversibility, bounds, and caps:

```
policies:
  - action_type: payments.transfer      # exact match on ActionRequest.action_type
    tier: 3                             # 0 observe | 1 auto | 2 auto-capped | 3 confirm
    dry_run: true                       # default true: new action types rehearse first
    compensating_command: payments.reverse   # REQUIRED for tiers 1 and 2
    undo_window_seconds: 900
    bounds:
      numeric:  { amount_eur: { min: 0.01, max: 500 } }
      enum:     { currency: [EUR] }
      required: [payee, amount_eur]
      strict_params: true               # reject undeclared params (injection defense)
    caps:
      daily_rate: 20                    # max executions per local day
      eur_day:   "50.00"                # tier-2 cumulative budgets (decimal strings)
      eur_month: "500.00"
```

Field notes that save you an afternoon

Field	What to know
dry_run / dry_run_until	A dry-run resolves *before* caps — rehearsals never spend budget. The two fields are ORed: dry_run: true rehearses indefinitely and a date will never switch it live; to rehearse until a date, set dry_run: false and give the date. Dry-run governs the automatic path only: a Tier-3 action in dry-run still creates a real approval, and approving executes for real.
compensating_command	The reversibility rule. Must name another registered action type; the undo runs through the same pipeline, linked to its parent. Every auto-executing tier needs one: a rule without it fails at load, and an auto action that loses its reversal demotes to Tier 3 at runtime.
bounds	Validated for every tier before proposals are created, so approvers only ever see sane requests. strict_params: true rejects any parameter not named.
caps + cost_param	daily_rate counts executions, reserved at decide time (race-free). Euro caps apply to Tier 2 and need to know which parameter carries the money: any action with a euro cap needs a cost_param (or a caller setting cost_eur). An amount the engine cannot resolve is a denial with reason cost_unknown — never an assumed zero.

Effects — governance that survives renaming

Policy binds to action *names*, but the same real-world effect is reachable through many tools (a payments tool, or a generic HTTP tool aimed at the bank). Effects give the engine a second, name-independent binding: label actions (or match parameters deterministically — a regex on a string, or canonicalized URL rules for hosts, subdomains, CIDRs) with an effect such as money.egress, then set a tier floor and one shared budget on the effect itself. Every carrier draws from the same cap; escalations carry reason effect_floor. A URL the canonicalizer cannot interpret is denied as malformed — a parse differential is a denial, never a bypass — and declared redirector hosts are never auto-executed: a human approves, or policy denies.

6 · The working loop

1 · Draft	Create a draft on the Drafts screen; add rules in the editor.
2 · Ratify	Review and ratify — the ceremony shows what will be in force before you confirm. You now have a numbered version and a receipt.
3 · Wire	Point your agents at the engine (§7). Every action flows through decide-and-reserve and returns a terminal result with a reason code.
4 · Watch	History fills with chain-numbered decisions; Live state shows budgets and the switch.
5 · Replay, then change	Before ratifying a stricter version, open past decisions and re-evaluate under your candidate's predecessors: would this have been refused? Test policy against reality, then draft the amendment and ratify n+1.
6 · Verify	Export any receipt's two files and check them offline (§8). That is what you hand an auditor.

First decision on a fresh install — the smallest honest way to put a row in History:

```
python -m onedoor.studio.walkthrough --db onedoor.db
```

7 · Wiring your agents — pick one door

You have	Use
A Python codebase (agent, framework, backend)	Embed the library — call decide_and_reserve in-process (docs: integration-library).
Anything that speaks HTTP	Run the decision service and POST /v1/decide (docs: integration-service).
An MCP host + tool servers	The MCP proxy — tools gated transparently (docs: integration-mcp).
A LiteLLM proxy	The custom guardrail adapter (docs: integration-litellm).
A LangGraph / LangChain agent	Governed tool wrapper + interrupts (docs: integration-langgraph).

Whichever door: *permitted* is an obligation (you enforce, then report the result), reserve-and-commit means a crashed action never leaks budget, and two concurrent requests can never share the last slot under a cap.

8 · Verifying a receipt — without trusting the Studio

On the Verify screen, open a receipt and save the two files it shows as receipt.json and snapshot.json. Then, on any machine:

```
python -m onedoor.studio.verify receipt.json snapshot.json
```

Output	Exit	Meaning
verified	0	The receipt matches its own digest and the snapshot hashes to the version the receipt ratified.
failed	1	One of those did not hold.
unreadable	2	A file could not be read. Not a failed check — a check that never ran.

The command reads two files and opens no database. Copy them to another machine and it gives the same answer — which is the entire point.

9 · Reason codes you will see

Code	Meaning
passed	Checks passed; the action was permitted.
default_deny	No policy for this action type — unknown actions cannot auto-execute.
tier_confirm	Tier 3: proposed, awaiting human approval.
no_compensating_command	An auto tier lost its reversal — demoted to confirm.
bounds	A parameter was out of bounds (or undeclared, under strict_params).
dry_run	Rehearsal: logged, nothing executed, no budget spent.
cap_rate / cap_value	A rate or money cap would be exceeded (window and unit in the budget object).
cost_unknown	A euro cap applies but the amount could not be resolved — denied, never assumed zero.
kill_switch	The switch is thrown; automatic execution blocked (Tier-0 reads exempt).
observe	Tier 0: recorded, allowed, no connector reached.
effect_floor	An effect label escalated the tier (e.g. money.egress carries a floor).
malformed	A target could not be canonicalized — a parse differential is a denial, never a bypass.
expired	An approval outlived its TTL.

10 · Design guarantees you can rely on

- An action type absent from the policy table **cannot** auto-execute (default-deny → Tier 3).
- A Tier-1 policy without a compensating_command **cannot** load, and cannot execute (runtime demotion to Tier 3).
- Bounds are validated before a proposal is created — approvers only ever see in-bounds requests.
- A dry-run never consumes caps.
- Two concurrent requests cannot share the last slot under a cap (check-and-reserve inside the deciding transaction).
- The audit log is append-only; results link to intents; an undo links to its parent.
- The Studio serves loopback only, and ratified versions are immutable — changes are new versions, never edits.

11 · When something looks wrong

Symptom	What it means
The Studio shows an empty world you know is not empty	It opened a different store than your service. Restart with --db naming the engine's database explicitly; the fresh-store warning names both defaults.
A rule refuses to load	Most often a tier-1/2 rule without a compensating_command — that is a boot failure by design, not a bug.
An action you allowed was denied cost_unknown	The euro cap could not find the amount: set cost_param on the rule (and list that parameter under bounds.required), or have the caller set cost_eur.

Symptom	What it means
A dry-run action “did nothing”	That is the contract: logged, no execution, no budget. For Tier 3, dry-run still proposes — approving executes for real.
verify prints unreadable	A file problem, not a verification failure. Re-export the two files and run again.
You need the Studio on another machine	You cannot — loopback only, no override flag. Use an SSH tunnel if you must view it remotely.

Write down anything that surprises you, in your own words, including things that merely felt wrong. The findings that shaped this Studio most came from someone using it and saying what happened.